PROGRAMMING LANGUAGES

HTML & CSS

Semantic markup, accessibility, and the modern CSS layout toolkit — box model, flexbox, grid, responsive design, and the cascade — built for real work and interviews.

html · css · frontend · accessibility · flexbox · css-grid · responsive-design · interview-prep

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. What HTML & CSS Are & Why They Matter
2. Setup & Getting Started
3. Semantic HTML & Document Structure
4. Forms & Input Controls
5. Accessibility (a11y)
6. The CSS Box Model
7. Flexbox — One-Dimensional Layout
8. CSS Grid — Two-Dimensional Layout
9. The Cascade, Specificity & Inheritance
10. Responsive Design & Media Queries
11. Custom Properties, Transitions & Modern CSS
12. Cheat-Sheet & Comparisons
13. Best Practices & Common Pitfalls
14. Interview Questions & Answers
15. Further Learning

What HTML & CSS Are & Why They Matter

HTML is the structure and meaning of a web document; CSS is its presentation. Every website, web app, email template, and electron desktop app ultimately renders through these two languages. Mastering them is not optional front-end polish — it is the foundation that determines accessibility, performance, SEO, and whether your JavaScript framework output is actually usable.

The browser builds a **DOM** (Document Object Model) from your HTML and a **CSSOM** (CSS Object Model) from your styles, combines them into a *render tree*, computes layout (positions and sizes), then paints pixels. Understanding this pipeline explains almost every rendering behaviour and performance characteristic you will ever debug.

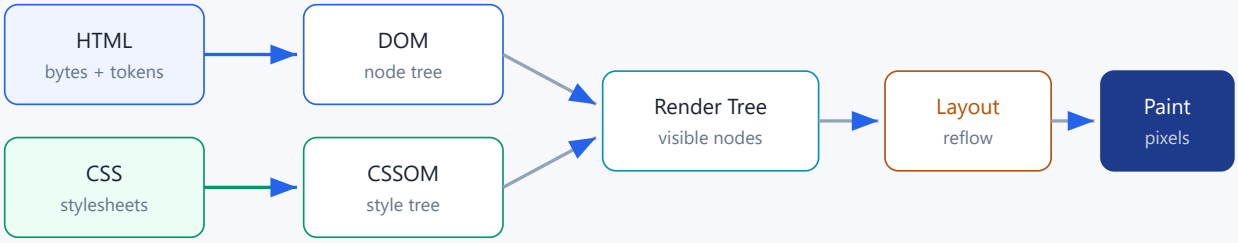


Fig 1 — The critical rendering path: HTML and CSS are parsed in parallel, merged, laid out, then painted.

Standards

HTML is the "Living Standard" maintained by the WHATWG — there is no "HTML6". CSS evolves as independent modules (Flexbox, Grid, Cascade Layers, Container Queries). As of 2026, baseline-safe modern CSS includes `:has()`, container queries, cascade layers, `subgrid`, and nesting.

Setup & Getting Started

No build tooling is required to start — a browser and a text editor are enough. For real projects you will add a dev server with live reload.

BASH

```
# Zero-tooling: just open the file
# create index.html and double-click it, or:
python -m http.server 8000        # serve current dir at http://localhost:8000

# Modern dev server with hot reload (Vite)
npm create vite@latest my-site -- --template vanilla
cd my-site && npm install && npm run dev

# Validate your HTML
npx html-validate index.html

# Lint & auto-format CSS
npx stylelint "**/*.css"
npx prettier --write "**/*.{html,css}"
```

Every HTML document starts from the same skeleton. Memorise it — interviewers notice when the boilerplate is wrong.

HTML

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <title>My Page</title>
    <link rel="stylesheet" href="styles.css" />
  </head>
  <body>
    <h1>Hello, world</h1>
    <script src="app.js" defer></script>
  </body>
</html>
```

✦ **Two non-negotiables**

`<!DOCTYPE html>` triggers standards mode (omitting it causes "quirks mode" layout bugs), and the `viewport` meta tag is required for responsive design to work on phones.

Semantic HTML & Document Structure

Semantic elements describe *meaning*, not appearance. A `<nav>` is a navigation region whether or not it looks like a menu. Semantics drive accessibility (screen readers announce landmarks), SEO (search engines weight headings), and maintainability. The anti-pattern — a page built entirely from `<div>` and `<span>` ("div soup") — works visually but is invisible to assistive tech.

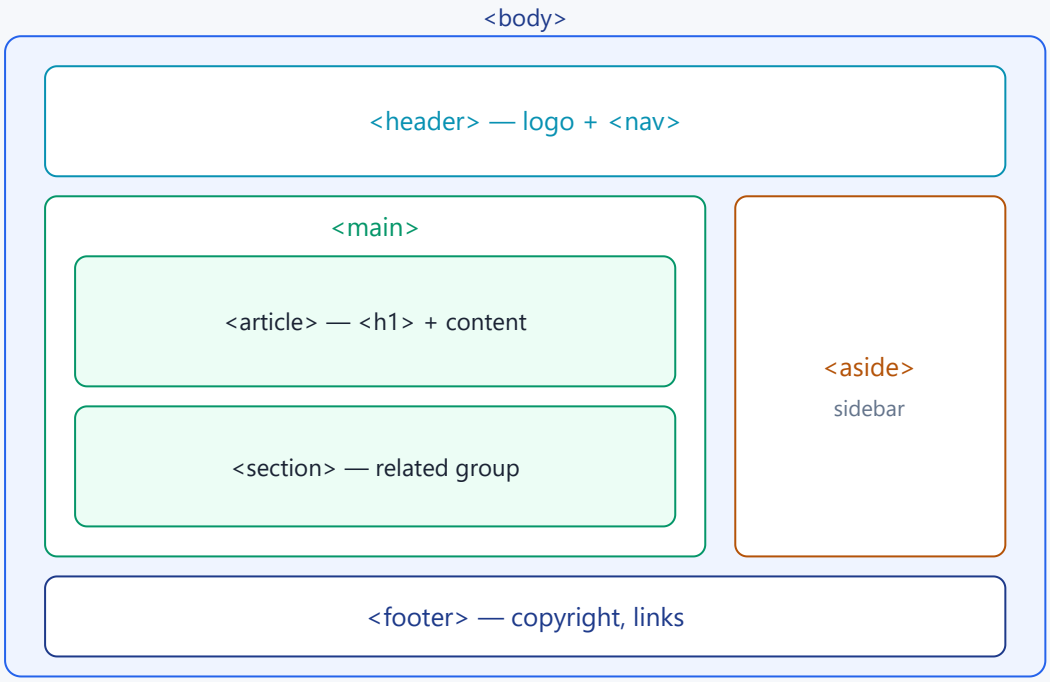


Fig 2 — A semantic page layout. Each landmark becomes a navigable region for screen readers.

Landmark & content elements

Element	Meaning	Use for
`<header>`	Introductory content	Site/section banner, logo, nav
`<nav>`	Navigation links	Primary menus, breadcrumbs
`<main>`	Dominant content (one per page)	The unique page content
`<article>`	Self-contained, reusable unit	Blog post, product card, comment
`<section>`	Thematic grouping with a heading	Chapters, tabbed panels
`<aside>`	Tangential content	Sidebars, pull quotes, ads
`<figure>/<figcaption>`	Self-referential media	Images/charts with captions
`<time>`	Machine-readable date	`datetime` attribute

HTML

```
<article>
  <header>
    <h2>Understanding the Cascade</h2>
    <p>By Ada Lovelace · <time datetime="2026-01-15">Jan 15, 2026</time></p>
  </header>
  <p>CSS resolves conflicts using origin, specificity, and order&hellip;</p>
  <figure>
    <img src="cascade.png" alt="Diagram showing how CSS rules are prioritised" />
    <figcaption>How the cascade picks a winning declaration.</figcaption>
  </figure>
</article>
```

⚠ **Headings are an outline**

Use `<h1>` – `<h6>` in order; never skip levels for visual size (use CSS for size). Exactly one `<h1>` per page describes the whole document. Screen-reader users navigate by heading level.

Forms & Input Controls

Forms are where accessibility, validation, and UX intersect. The single most important rule: **every input needs a label**, programmatically associated via `for` / `id`.

HTML

```html
<form action="/signup" method="post">
  <!-- Label associated by matching for/id -->
  <label for="email">Email address</label>
  <input
    type="email"
    id="email"
    name="email"
    autocomplete="email"
    required
    placeholder="you@example.com"
  />

  <label for="pw">Password</label>
  <input type="password" id="pw" name="password"
         minlength="8" required />

  <!-- Group related controls -->
  <fieldset>
    <legend>Plan</legend>
    <label><input type="radio" name="plan" value="free" checked /> Free</label>
    <label><input type="radio" name="plan" value="pro" /> Pro</label>
  </fieldset>

  <button type="submit">Create account</button>
</form>
```

Use the right `type` — mobile keyboards and built-in validation depend on it. Pair `name` (sent to the server) with `autocomplete` (lets the browser/password manager help the user).

Input type	Purpose	Mobile keyboard
`email`	Email with built-in validation	@ key shown
`tel`	Phone numbers	Numeric dial pad
`number`	Numeric with spinners	Number pad
`date` / `time`	Native date/time picker	Picker UI
`search`	Search box (clearable)	Search/Go key
`url`	URL with validation	.com / slash keys
`range`	Slider	—
`color`	Native colour picker	—

✦ **Free native validation**

Attributes `required`, `min` / `max`, `minlength` / `maxlength`, `pattern`, and `type` give you client-side validation with zero JS. Style states with the `:valid`, `:invalid`, and `:user-invalid` pseudo-classes. Always re-validate on the server — client validation is UX, not security.

Accessibility (a11y)

Accessibility means your site works for everyone — keyboard-only users, screen-reader users, people with low vision or motor impairments. It is also a legal requirement in many jurisdictions (WCAG 2.2 AA is the common target) and overlaps heavily with good SEO.

PERCEIVABLE

Text alternatives (`alt`), captions, sufficient colour contrast (4.5:1 for body text).

OPERABLE

Everything reachable and usable by keyboard; visible focus; no keyboard traps.

UNDERSTANDABLE

Clear labels, predictable behaviour, helpful error messages.

Core techniques

HTML

```html
<!-- 1. Meaningful alt text (empty alt="" for decorative images) -->
<img src="chart.png" alt="Revenue grew 40% from Q1 to Q4" />
<img src="divider.svg" alt="" />

<!-- 2. Use native buttons/links, not divs with onclick -->
<button type="button">Open menu</button>   <!-- focusable + keyboard by default -->

<!-- 3. ARIA only when HTML cannot express it -->
<nav aria-label="Breadcrumb"> ... </nav>
<button aria-expanded="false" aria-controls="menu">Menu</button>

<!-- 4. Visually-hidden text for screen readers -->
<a href="/cart">
  <svg aria-hidden="true"> ... </svg>
  <span class="sr-only">View shopping cart</span>
</a>

<!-- 5. Skip link as the first focusable element -->
<a href="#main" class="skip-link">Skip to content</a>
```

CSS

```css
/* The canonical "visually hidden but screen-reader readable" utility */
.sr-only {
  position: absolute;
  width: 1px; height: 1px;
  padding: 0; margin: -1px;
  overflow: hidden;
  clip: rect(0, 0, 0, 0);
  white-space: nowrap;
  border: 0;
}
/* Never remove focus outlines without a replacement */
:focus-visible { outline: 3px solid #2563eb; outline-offset: 2px; }
```

First rule of ARIA

No ARIA is better than bad ARIA. A native `<button>` beats `<div role="button" tabindex="0">` every time — you get focus, keyboard activation, and semantics for free. Reach for ARIA only to fill genuine gaps (live regions, custom widgets).

The CSS Box Model

Every element is a rectangular box composed of four layers: content, padding, border, and margin. Misunderstanding how width is calculated is the single most common beginner CSS bug.

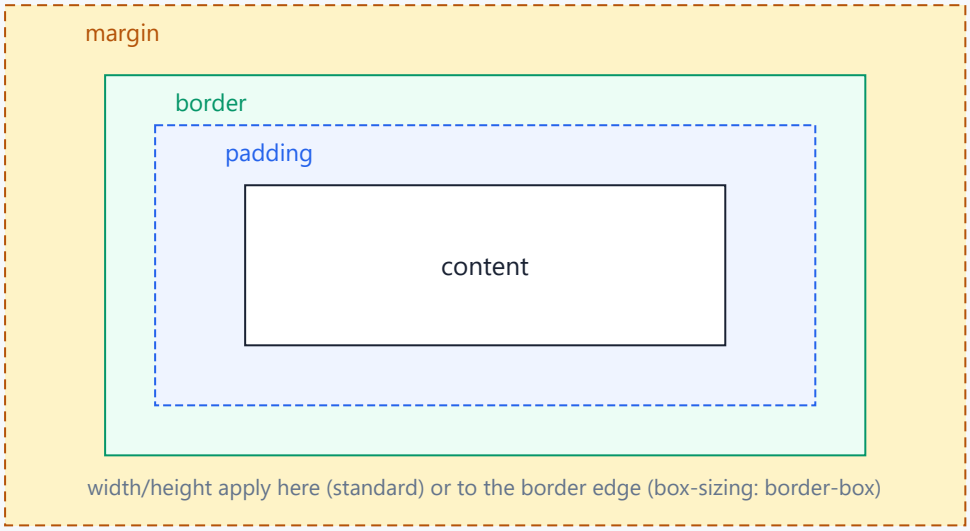


Fig 3 — The box model. With `box-sizing: border-box`, declared width includes padding and border.

css

```css
/* The universal reset nearly everyone uses in 2026 */
*, *::before, *::after {
  box-sizing: border-box;   /* width now includes padding + border */
}

.card {
  width: 300px;         /* total rendered width stays 300px … */
  padding: 20px;        /*  … even with 20px padding */
  border: 2px solid #ccc;
  margin: 16px;         /* space OUTSIDE the box, transparent */
}
```

⚠ **Margin collapse**

Vertical margins between adjacent block elements *collapse* to the larger of the two (not the sum). This surprises everyone. It does not happen in flex/grid containers or across padding/borders — one reason modern layouts prefer `gap` over margins.

Flexbox — One-Dimensional Layout

Flexbox lays out items along a single axis (a row or a column) and excels at distributing space and aligning content. It is the right tool for navbars, toolbars, card rows, and centering.

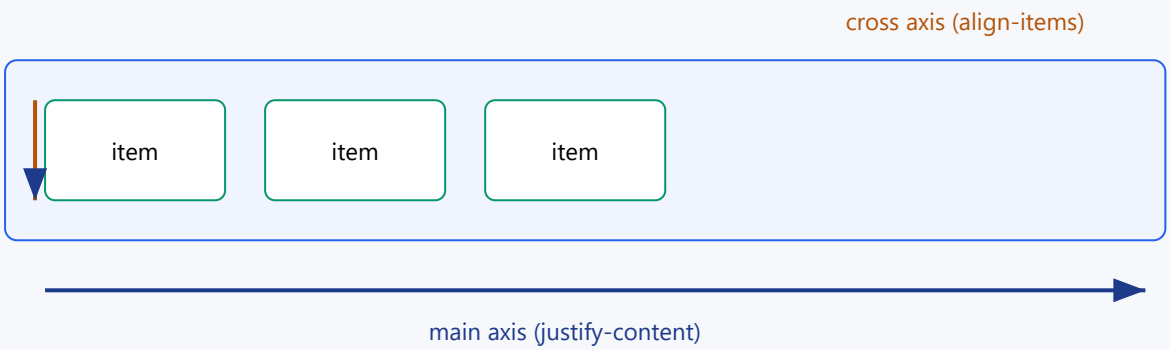


Fig 4 — Flexbox: items flow along the main axis; alignment on the cross axis.

CSS

```css
.navbar {
  display: flex;
  justify-content: space-between; /* main-axis distribution */
  align-items: center;            /* cross-axis alignment */
  gap: 1rem;                      /* spacing between items */
}

/* Perfect centering — the classic interview question */
.center {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}

/* Flexible item sizing: grow shrink basis */
.sidebar { flex: 0 0 240px; }  /* fixed 240px, never grow/shrink */
.content { flex: 1; }          /* takes all remaining space */
```

Property	On	Does
`flex-direction`	container	`row` (default) / `column`
`justify-content`	container	Main-axis: start, center, space-between, space-around
`align-items`	container	Cross-axis: stretch, center, flex-start, baseline
`flex-wrap`	container	Allow items to wrap to new lines
`gap`	container	Gutter between items
`flex`	item	Shorthand: grow shrink basis
`align-self`	item	Override cross-axis alignment per item

CSS Grid — Two-Dimensional Layout

Grid controls rows *and* columns simultaneously. Use it for page skeletons, dashboards, and any genuinely two-dimensional layout. Flexbox and Grid are complementary, not competitors — Grid for the overall scaffold, Flexbox for the components inside each cell.

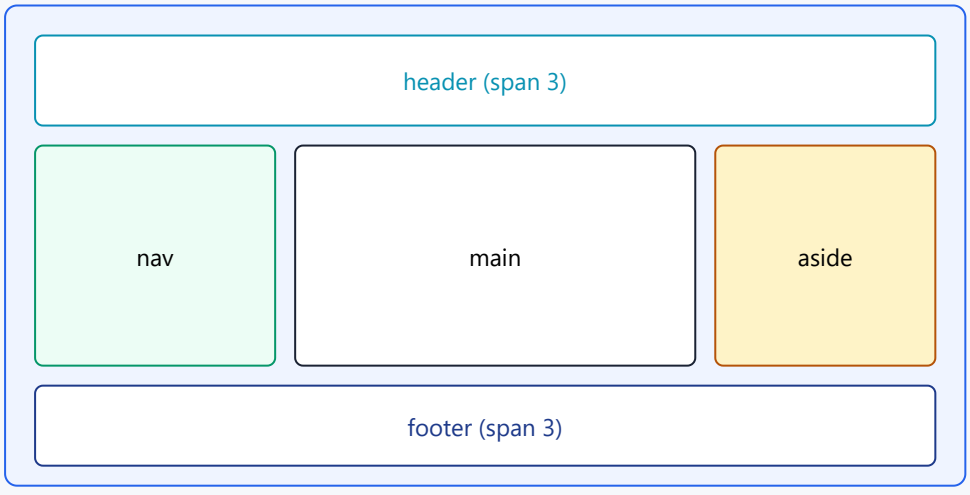


Fig 5 — A classic "holy grail" layout expressed with named grid areas.

```css
```

```css
.layout {
  display: grid;
  grid-template-columns: 160px 1fr 200px;  /* fr = fraction of free space */
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header header header"
    "nav    main   aside"
    "footer footer footer";
  gap: 1rem;
  min-height: 100vh;
}
header { grid-area: header; }
nav    { grid-area: nav; }
main   { grid-area: main; }
aside  { grid-area: aside; }
footer { grid-area: footer; }

/* Responsive auto-fitting card grid — no media queries needed! */
.cards {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 1.5rem;
}
```

✦ **The one-liner you will reuse forever**

`repeat(auto-fit, minmax(240px, 1fr))` creates a responsive grid that fits as many 240px-min columns as will fit, stretching them to fill the row. It reflows automatically with the viewport — zero media queries.

The Cascade, Specificity & Inheritance

When multiple rules target the same element, CSS resolves the winner through a well-defined algorithm. Interviewers love this because misunderstanding it causes the dreaded "why won't this style apply?" and the `!important` arms race.

Resolution order (highest wins): **origin & importance** → **cascade layers** → **specificity** → **source order**. Specificity is counted as a tuple `(inline, IDs, classes/attributes/pseudo-classes, elements)`.

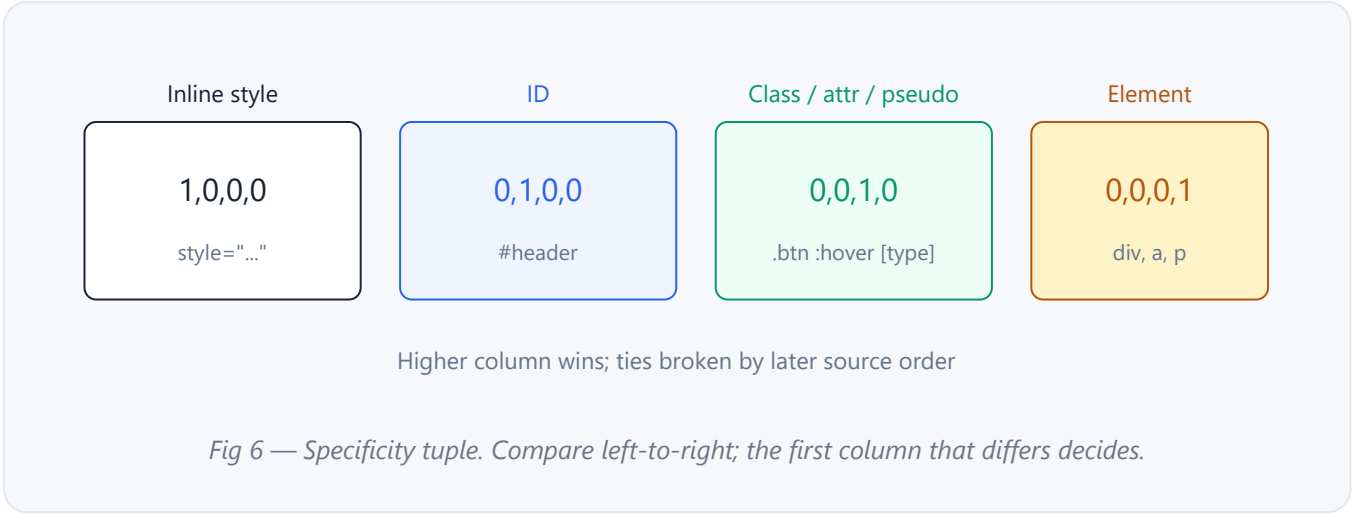


Fig 6 — Specificity tuple. Compare left-to-right; the first column that differs decides.

css

```css
/* Specificity examples */
p            { color: black; }   /* 0,0,0,1 */
.intro       { color: blue;  }   /* 0,0,1,0  -- wins over p */
#main .intro { color: green; }   /* 0,1,1,0  -- wins over .intro */

/* Cascade layers tame specificity wars across large codebases */
@layer reset, base, components, utilities;
@layer components { .btn { color: blue; } }
@layer utilities  { .text-red { color: red; } } /* later layer wins */

/* Modern specificity escape hatch without raising specificity */
:where(.a, .b) { color: gray; } /* :where() contributes 0 specificity */
```

Avoid `!important`

It overrides normal specificity and starts an escalation war that becomes unmaintainable. Prefer cascade layers, lower-specificity selectors, and `:where()`. Reserve `!important` for overriding third-party inline styles you cannot edit.

Responsive Design & Media Queries

Responsive design means one codebase adapts to any screen. The modern approach is **mobile-first**: write the base styles for small screens, then layer enhancements with `min-width` media queries as space allows.

css

```css
/* Mobile-first base */
.container { padding: 1rem; }
.grid { display: grid; gap: 1rem; grid-template-columns: 1fr; }

/* Tablet and up */
@media (min-width: 48rem) {       /* ~768px; prefer rem over px */
  .grid { grid-template-columns: repeat(2, 1fr); }
}
/* Desktop and up */
@media (min-width: 64rem) {       /* ~1024px */
  .grid { grid-template-columns: repeat(3, 1fr); }
  .container { max-width: 72rem; margin-inline: auto; }
}

/* Respect user preferences */
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after { animation: none !important; transition: none !important; }
}
@media (prefers-color-scheme: dark) {
  :root { --bg: #0b1020; --fg: #e5e7eb; }
}

/* Container queries: style based on the PARENT'S size, not the viewport */
.card-wrap { container-type: inline-size; }
@container (min-width: 30rem) {
  .card { display: grid; grid-template-columns: 120px 1fr; }
}
```

Prefer fluid, intrinsic techniques that reduce the number of breakpoints you must maintain:

FLUID TYPE

`clamp(1rem, 0.5rem + 2vw, 1.5rem)` scales font size smoothly between a min and max.

LOGICAL UNITS

`min()`, `max()`, `clamp()`, and `vw/vh/dvh` adapt without breakpoints. `dvh` handles mobile browser chrome.

Container queries changed everything

As of 2026 they are baseline-supported. A component can now respond to the space *it* occupies, making truly reusable components possible — the same card adapts whether it sits in a wide main column or a narrow sidebar.

Custom Properties, Transitions & Modern CSS

CSS custom properties (variables) cascade and can be read/written at runtime with JavaScript — the backbone of theming and design systems.

CSS

```css
:root {
  --brand: #2563eb;
  --radius: 8px;
  --space: 1rem;
  --fg: #1a2233;
}
[data-theme="dark"] {    /* override by toggling an attribute on <html> */
  --brand: #60a5fa;
  --fg: #e5e7eb;
}
.btn {
  background: var(--brand);
  border-radius: var(--radius);
  padding: calc(var(--space) / 2) var(--space);
  color: white;
  transition: transform 150ms ease, background 150ms ease;
}
.btn:hover { transform: translateY(-2px); background: color-mix(in srgb, var(--brand)

/* Nesting (baseline 2024+) keeps related rules together */
.card {
  padding: var(--space);
  & > h3 { margin-top: 0; }
  &:hover { box-shadow: 0 4px 12px rgb(0 0 0 / 0.1); }
}

/* :has() — the long-awaited "parent selector" */
.form:has(input:invalid) .submit { opacity: 0.5; pointer-events: none; }
```

Transitions vs animations

`transition` animates between two states (e.g. hover). `@keyframes` + `animation` defines multi-step sequences that can loop and run on load. For performance, animate only `transform` and `opacity` — they are GPU-composited and skip layout/paint.

Mini worked example: an accessible theme-toggle button

HTML

```
<button id="themeBtn" aria-pressed="false">
  <span class="sr-only">Toggle dark mode</span>🌓
</button>
<script>
  const btn = document.getElementById('themeBtn');
  btn.addEventListener('click', () ⇒ {
    const dark = document.documentElement.toggleAttribute('data-dark');
    btn.setAttribute('aria-pressed', String(dark));
  });
</script>
```

CSS

```
:root        { --bg: #ffffff; --fg: #1a2233; }
[data-dark]  { --bg: #0b1020; --fg: #e5e7eb; }
body { background: var(--bg); color: var(--fg); transition: background .2s, color .2s;
```

Cheat-Sheet & Comparisons

Units quick reference

Unit	Relative to	Best for
`px`	Absolute pixel	Borders, fine detail
`rem`	Root font-size	Font sizes, spacing (scales with user prefs)
`em`	Element's font-size	Padding relative to text
`%`	Parent dimension	Fluid widths
`fr`	Grid free space	Grid tracks
`vw/vh`	Viewport	Full-screen sections
`dvh`	Dynamic viewport	Mobile full-height (accounts for toolbars)
`ch`	Width of "0"	Readable line length (~60-75ch)

Flexbox vs Grid

Aspect	Flexbox	Grid
Dimensions	One axis at a time	Two axes together
Content vs layout	Content-driven (sizes from items)	Layout-driven (defines the structure)
Best for	Navbars, toolbars, chip rows, centering	Page skeletons, dashboards, galleries
Overlap	Harder	Easy (cells can overlap)

Position values

Value	Behaviour
`static`	Default; normal flow
`relative`	Offset from its normal spot; creates a positioning context
`absolute`	Removed from flow; positioned vs nearest positioned ancestor
`fixed`	Positioned vs viewport; stays on scroll
`sticky`	Relative until a scroll threshold, then fixed

Best Practices & Common Pitfalls

✓ **Do**

Start semantic and mobile-first. Use `box-sizing: border-box` globally. Prefer `gap` over margins for spacing in flex/grid. Keep selectors flat and low-specificity. Use `rem` for type/spacing, `px` for borders. Test with a keyboard and a screen reader.

✦ **Do**

Centralise design tokens in custom properties. Use logical properties (`margin-inline` , `padding-block`) for internationalisation. Add `loading="lazy"` to below-the-fold images and always set `width` / `height` to prevent layout shift (CLS).

⚠ **Watch out**

Fixed heights cause overflow with real content — prefer `min-height` . `100vh` is taller than the visible area on mobile; use `100dvh` . Forgetting the viewport meta tag breaks all responsiveness. Removing focus outlines without a replacement fails WCAG.

✕ **Avoid**

Div soup, inline styles, deeply nested selectors, ID selectors for styling, and the `!important` arms race. Never convey information by colour alone (colour-blind users). Never use `<table>` for page layout.

Interview Questions & Answers

What is the difference between `<section>` and `<div>`?

`<section>` is semantic — it denotes a thematic grouping of content that typically has a heading and may appear in the document outline. `<div>` is a generic, meaningless container used purely for styling or scripting hooks. Reach for `<div>` only when no semantic element fits.

Explain the CSS box model and `box-sizing`.

Each element is content + padding + border + margin. With the default `content-box`, `width` sets only the content width, so padding/border are added on top. With `border-box`, `width` includes padding and border, so a 300px box stays 300px — far more intuitive, which is why the universal `* { box-sizing: border-box }` reset is standard.

How is CSS specificity calculated?

As a tuple (inline, IDs, classes/attributes/pseudo-classes, elements). Compare left to right; the first differing column decides. Inline styles beat IDs, which beat classes, which beat element selectors. Ties are broken by source order (later wins). `!important` and cascade layers sit above this.

When would you choose Flexbox over Grid?

Flexbox for one-dimensional, content-driven layouts where items size from their content and distribute along a single axis — navbars, button rows, centering. Grid for two-dimensional layouts where you define the structure of rows and columns up front — page scaffolds, dashboards, image galleries. They compose well: Grid for the page, Flexbox inside cells.

What does `position: sticky` do and what are its gotchas?

It behaves like `relative` until the element reaches a scroll offset (set via `top` / `bottom`), then sticks like `fixed` within its scroll container. Gotchas: it needs a threshold value to work, it is confined to its parent's bounds, and an ancestor with `overflow: hidden/auto` silently breaks it.

How do you center a div both horizontally and vertically?

Modern answer: `display: grid; place-items: center;` on the parent, or flexbox with `justify-content: center; align-items: center;`. For absolute positioning: `position: absolute; inset: 0; margin: auto;` on a sized box, or `top/left: 50%; transform: translate(-50%, -50%);`.

What is margin collapsing?

Adjacent vertical margins between block-level elements combine into one margin equal to the larger of the two, not their sum. It occurs between siblings and between parent/first-or-last child. It

does not happen in flex/grid containers, or when separated by padding, border, or content — which is why `gap` is predictable.

How do you make a site accessible?

Use semantic HTML and a correct heading outline; label every form control; provide meaningful `alt` text; ensure full keyboard operability with visible focus; meet 4.5:1 contrast; never convey meaning by colour alone; use ARIA only to fill genuine gaps; and test with a keyboard and a screen reader (NVDA/VoiceOver). Target WCAG 2.2 AA.

What is the difference between `rem` and `em`?

`rem` is always relative to the root (`<html>`) font-size, so it is predictable and respects the user's browser setting. `em` is relative to the current element's font-size and compounds through nesting, which can cause runaway sizing. Use `rem` for type and spacing; `em` when you want something to scale with its own text (e.g. padding on a button).

What are container queries and why do they matter?

Container queries style an element based on the size of its container rather than the viewport. This makes components truly reusable — the same card can switch to a horizontal layout when it has room, regardless of where it sits on the page. They complement media queries (which remain right for page-level breakpoints) and are baseline-supported in 2026.

How do you prevent Cumulative Layout Shift (CLS)?

Always set explicit `width` / `height` (or `aspect-ratio`) on images and media so the browser reserves space; reserve space for ads/embeds; avoid inserting content above existing content; and use `font-display: optional/swap` plus preloading to limit font-swap shift. CLS is a Core Web Vital.

What is the difference between `visibility: hidden`, `display: none`, and `opacity: 0`?

`display: none` removes the element from the layout and the accessibility tree (not focusable, takes no space). `visibility: hidden` keeps its space but makes it invisible and non-interactive. `opacity: 0` keeps space *and* interactivity — it is still clickable and focusable, which is a common bug.

Further Learning

- **MDN Web Docs** — the authoritative reference for every HTML element and CSS property, with live examples and baseline support data.
- **web.dev (Learn CSS / Learn HTML / Learn Accessibility)** — Google's structured, modern courses.

- **WCAG 2.2 & the WAI-ARIA Authoring Practices Guide (APG)** — patterns for accessible widgets.
- **CSS Grid & Flexbox games** — Grid Garden and Flexbox Froggy for muscle memory.
- **Practice projects** — Frontend Mentor challenges; rebuild a real site's layout from scratch using only semantic HTML and modern CSS (Grid + custom properties, no framework).
- **Next steps** — a CSS methodology (BEM or utility-first like Tailwind), build tooling (Vite, PostCSS), and a component framework (React/Svelte) once the fundamentals are solid.